

Clash Verge Rev 全局链式代理脚本使用说明

这个目录里的 `Script.js` 是 Clash Verge Rev 的全局增强脚本模板。它会在每个订阅生成最终 Mihomo 配置时自动执行，所以换订阅后不用重新改脚本。

链式代理方向

脚本里的链路方向是：

本机应用 -> 订阅节点/前置节点 -> 可选中转节点 -> SOCKS5 落地节点 -> 目标网站

对应到配置名：

Chain-Front -> US-Chicago-Chain -> 目标网站

US-Chicago-Chain 里会自动生成：

dialer-proxy: Chain-Front

这表示 US-Chicago-Chain 会通过 Chain-Front 拨出。Chain-Front 里默认包含当前订阅里的 节点选择、自动选择和 DIRECT。

三级跳时方向类似：

Chain-Front -> US-NewYork-Hop -> US-Chicago-Chain -> 目标网站

最后一个 hop 才是最终落地节点，中间 hop 只是中转。

能不能 GUI 直接导入 JS?

Clash Verge Rev 当前更准确的用法不是“把 JS 当订阅导入”。订阅导入框主要处理 HTTP/HTTPS 订阅链接，不是导入增强脚本文件。

推荐两种方式：

1. GUI 粘贴法，适合给别人用。
2. 直接覆盖配置目录里的 `Script.js`，适合熟悉文件路径的人。

方法一：GUI 粘贴法

1. 打开 Clash Verge Rev。
2. 进入 Profiles / 配置 页面。
3. 找到全局增强项 Script / Global Script / 全局脚本。
4. 双击或右键选择编辑。
5. 打开本目录的 `Script.js`，复制全部内容，粘贴覆盖编辑器里的内容。
6. 保存。
7. 刷新或切换一次订阅，让配置重新生成。

源码确认：Clash Verge Rev 会先执行全局 `Script.js`，再执行当前订阅自己的扩展脚本。因此全局 `Script.js` 对切换后的订阅仍然生效。

方法二：直接覆盖文件

Linux 上配置目录通常是：

```
~/.local/share/io.github.clash-verge-rev.clash-verge-rev/profiles/Script.js
```

把本目录的 `Script.js` 覆盖到上面的路径，然后回到 Clash Verge Rev 里刷新配置或重启 Clash Verge Rev。

如果不知道配置目录在哪，可以在 Clash Verge Rev 的全局 `Script` 项上右键，选择打开文件，然后替换打开的那个文件内容。

必须先改的地方

打开 `Script.js`，先改 `CHAINS` 里的 SOCKS5 落地节点：

```
{
  name: "US-Chicago",
  chainName: "US-Chicago-Chain",
  chainGroup: "Chain-US-Chicago",
  hops: [
    {
      name: "US-Chicago",
      proxy: {
        type: "socks5",
        server: "YOUR_LANDING_SERVER",
        port: 443,
        username: "YOUR_USERNAME",
        password: "YOUR_PASSWORD",
        udp: true,
      },
    },
  ],
},
]
```

如果你的 SOCKS5 不需要用户名密码，可以删掉 `username` 和 `password` 两行。

换订阅后会怎样

不需要改脚本。

换订阅后，脚本会重新根据新订阅生成这些组：

- `Chain-Front`：前置节点选择组，引用新订阅里的 `节点选择`、`自动选择`、`DIRECT`。
- `US-Chicago`：直连 SOCKS5 落地节点。
- `US-Chicago-Chain`：后置落地节点，通过 `Chain-Front` 拨出。
- `Chain-US-Chicago`：链式落地选择组。
- `Domestic-Sites`：国内网站策略组，默认先 `DIRECT`，也保留落地节点选项。

- `Default`：最终默认出口。

新增落地节点

只改 `CHAINS` 数组，不要到处复制配置。例子：

```
{
  name: "CN-Shanghai",
  chainName: "CN-Shanghai-Chain",
  chainGroup: "Chain-CN-Shanghai",
  hops: [
    {
      name: "CN-Shanghai",
      proxy: {
        type: "socks5",
        server: "YOUR_CN_SERVER",
        port: 443,
        username: "YOUR_USERNAME",
        password: "YOUR_PASSWORD",
        udp: true,
      },
    },
  ],
}
```

再加一个美国纽约：

```
{
  name: "US-NewYork",
  chainName: "US-NewYork-Chain",
  chainGroup: "Chain-US-NewYork",
  hops: [
    {
      name: "US-NewYork",
      proxy: {
        type: "socks5",
        server: "YOUR_US_SERVER",
        port: 443,
        username: "YOUR_USERNAME",
        password: "YOUR_PASSWORD",
        udp: true,
      },
    },
  ],
}
```

命名约定：

- `US-Chicago` 表示落地节点本身。
- `US-Chicago-Chain` 表示通过前置节点再连这个落地节点。

- `Chain-US-Chicago` 表示给界面选择用的链式组。

三级跳或更多跳

在同一个 chain 的 `hops` 里放多个节点即可。脚本会按顺序自动生成 `dialer-proxy`。

例如：

本机 -> 订阅节点 -> US-NewYork-Relay -> US-Chicago -> 目标网站

对应配置：

```
{
  name: "US-Chicago-via-NewYork",
  chainName: "US-Chicago-via-NewYork-Chain",
  chainGroup: "Chain-US-Chicago-via-NewYork",
  hops: [
    {
      name: "US-NewYork-Relay",
      proxy: {
        type: "socks5",
        server: "YOUR_RELAY_SERVER",
        port: 443,
        username: "YOUR_USERNAME",
        password: "YOUR_PASSWORD",
        udp: true,
      },
    },
    {
      name: "US-Chicago",
      proxy: {
        type: "socks5",
        server: "YOUR_LANDING_SERVER",
        port: 443,
        username: "YOUR_USERNAME",
        password: "YOUR_PASSWORD",
        udp: true,
      },
    },
  ],
}
```

生成后的链路是：

Chain-Front -> US-Chicago-via-NewYork-Hop1-US-NewYork-Relay -> US-Chicago-via-NewYork-Chain

要四级跳、五级跳，就继续往 `hops` 里加节点。数组里最后一个 hop 是最终落地。

微信图片慢的处理

脚本里已经加了微信/QQ 相关直连规则和 TUN 排除：

- `find-process-mode: always`
- 微信/QQ 进程直连
- `qq.com`、`tencent.com`、`gting.com`、`qplic.cn` 等域名直连
- 部分微信图片相关 IP 段加入 `tun.route-exclude-address`

如果某个网络环境下仍然慢，先看 Clash Verge Rev 日志里微信图片请求命中了哪个规则，再补对应域名或 IP 段。

注意

不要把带真实 `server`、`username`、`password` 的脚本公开发布。给别人用时，让对方填自己的 SOCKS5 落地节点信息。